

What changed in version 0.2.0-beta.28

Six phases of planned improvements, plus a full-product quality sweep. This document states, for each change, **what the product did before** and **what it does now** — nothing else.

VERSION	0.2.0-beta.28 (previous: 0.2.0-beta.27)
DATE	24 July 2026
SCOPE	Phases 0–5 of the enhancement roadmap + whole-product QA sweep
BREAKING CHANGES	None. Two things were renamed; every old name still works.
VERIFICATION	1,963 automated tests passing · every phase validated on a real machine before the next began

The one-sentence version. Coodra gained four new command groups and a codebase-graph section in the dashboard; three protections that looked like they were working turned out not to be, and were fixed; and the product's own documentation was corrected where it had drifted away from the code.

At a glance

One line per change. Each is expanded in the pages that follow.

PHASE	AREA	BEFORE → NOW	TYPE
Phase 0	Policy enforcement	Most projects had no safety policy, so agents could edit protected files unchecked → every project is protected from the moment it is created.	FIX
Phase 0	Uninstall & restore	Uninstall left background services running; database restore could run over a live database → both now account for every service.	FIX
Phase 1	Agent setup	Connecting a coding agent meant re-running full setup and accepting what it auto-detected → add, check, repair, or remove one agent at a time.	NEW
Phase 2	Generated files	Coodra wrote files into a repository with no record of them; cleanup was manual guesswork → a tracked ledger and a safe cleanup command.	NEW
Phase 3	Codebase graph	No way to build or view a graph from Coodra, and on any real-sized repository no interactive graph was ever produced → build, view, and browse it in the dashboard.	NEW
Phase 4	Deep Wiki	Wiki generation only knew the file names → it now also knows the codebase structure and every decision previously recorded.	IMPROVED
Phase 5	Naming	Two different things were both called “Features” → the on-demand ones are now “Skills”. Old names still work.	RENAMED
QA sweep	Live sessions	A working session longer than 30 minutes was cancelled as “stuck”, and a second terminal killed the first one's session → activity is now what decides.	FIX
QA sweep	Reachability	The command menu listed six command groups that then refused to run → everything listed now runs.	FIX
QA sweep	Documentation	Instructions told AI agents to call tools that no longer existed under those names → 41 corrections across the docs.	FIX

Reliability defects

Four problems found by testing the shipped product on a real machine rather than reading the code. The first is the most serious thing in this release.

Safety policies were not being applied to most projects

BEFORE	NOW
Coodra's headline protection — blocking an AI agent from editing sensitive files such as <code>.env</code> — depended on a default policy being present for the project. On the validation machine, only 4 of 55 projects had one. For the other 51, the agent was allowed to write to those files. The feature was advertised, the audit trail looked normal, and nothing was actually being enforced.	A default policy is now created at both points where a project can come into existence, so a project cannot exist without one. A repair command backfills projects created before this fix. On the validation machine: 51 unprotected projects → 0 , and the built-in health check for enforcement went from failing to passing.

Uninstall left services running

BEFORE	NOW
Uninstalling removed Coodra's files but left its four background services running — including the dashboard, which kept holding its network port. There was also no way to remove the collected data.	Uninstall stops and removes all four services before deleting anything, and a new option removes the local database as well (configuration and saved knowledge are kept).

Database restore could overwrite a live database

BEFORE	NOW
Restore refused to run while services were active, but it only detected services it had started itself. Services started by the operating system's service manager — the normal case — were invisible to the check, so a restore could proceed over a database still in use.	The pre-flight check inspects every service through both mechanisms, so a restore is refused whenever anything is actually running.

The dashboard displayed a wrong version number

BEFORE	NOW
A fixed value was written into the page, so the dashboard reported a version the software had not been for some time.	The dashboard reports the version actually installed.

Connecting coding agents

Coodra works by connecting to the AI coding tools a developer already uses. Managing those connections was previously all-or-nothing.

Agents can now be managed one at a time

BEFORE	NOW
The only way to connect an agent was to re-run the full setup, which auto-detected whatever was installed and configured all of it. There was no way to: • add an agent you installed later, • repair one whose configuration had drifted, • remove one agent without disturbing the others, • or see which agents were correctly wired.	Four new commands — <code>add</code>, <code>status</code>, <code>repair</code>, <code>remove</code> — each accepting a single agent, all detected agents, or all supported ones: • Claude Code, Cursor, Codex, Windsurf, and Devin. • <code>status</code> reports each configuration file as present, missing, or drifted. • <code>repair</code> rewrites a drifted configuration back to correct. Initial setup now runs through the same mechanism, so the two can no longer disagree with each other.

Devin is recognised under its other product names as well, so a user need not know which one Coodra records internally.

Files Coodra writes into your repository

Coodra generates configuration and data files inside a project. Previously it kept no record of which ones were its own.

Generated files are now tracked, and cleanup is safe

BEFORE	NOW
Files were written across the repository with no inventory. To clean up, a user had to know which files Coodra had created and delete them by hand — with no way to tell a disposable generated file from one containing their own edits. In practice, files were left behind.	A project-local <code>.coodra/</code> folder holds the project's identity and a ledger of every file Coodra generated, recording who owns it and whether it is safe to delete. • Status lists every tracked file and its category. • Clean is safe by default: it removes only files known to be disposable. Files that may contain your own edits require explicit confirmation, and shared machine-level configuration is never touched.

The previous identity file is still written alongside the new one, so nothing that reads the old location breaks.

Codebase graph

Coodra can use a knowledge graph of a repository — its files, symbols, and the calls between them — so an agent can answer “what does this change affect?”. The graph is produced by an external tool; Coodra manages and displays it.

Building and viewing a graph is now part of Coodra

BEFORE	NOW
Coodra could connect an agent to the graph tool, but not manage the result. There was no command to build a graph, no fixed location for the output, no way to open it, and nothing in the dashboard.	Three commands — <code>build</code> , <code>open</code> , <code>clean</code> — with output stored in a managed location that every part of Coodra agrees on. A new Codebase graph section in the dashboard lists each project's graph status, renders the structural report, and embeds the interactive graph.

No interactive graph was ever produced for a real repository

BEFORE	NOW
The underlying tool skips its interactive visualisation above 5,000 nodes and does not report having done so. Any codebase of consequence exceeds that, so the visual graph silently never existed — found while building this product's own graph of 13,982 nodes.	When the graph is too large for the standard visualisation, Coodra automatically produces a grouped overview instead — clusters of related code rather than individual files — so a graph is always viewable. This can be switched off for automated builds where nobody opens it.

An existing graph is never moved without asking

BEFORE	NOW
Not applicable — Coodra did not manage graph locations.	If a project already keeps its graph in the tool's default location — which teams commit to version control — Coodra leaves it there. Migration to the managed location only happens when the user agrees to it.

Deep Wiki

Deep Wiki is an explanation of a codebase — sections, pages, and diagrams — written by the user's own AI agent. Coodra supplies the background material the agent works from.

The wiki is now written with knowledge of structure and history

BEFORE	NOW
The agent was given a listing of files and little else. It had to infer the architecture from names alone, and had no access to decisions the team had already recorded — so a new wiki could describe the codebase in terms that contradicted earlier choices.	The background material now includes: • the largest clusters of related code, as candidate wiki sections; • the most connected files, as candidates for the most important pages; • a summary of the codebase's structural report; • the project's previously recorded decisions and session summaries. On this product's own repository that supplied 769 code clusters, 18 recorded decisions, and 9 session summaries.

Two silent gaps in that background material

BEFORE	NOW
The wiki looked for graph data in one fixed location and missed it entirely for projects using the managed location. It also read the wrong field when counting connections, so that figure was always blank. Neither failure produced a warning — the wiki was simply written with less information than was available.	Both corrected. The graph is found wherever the project keeps it, and the connection count is real.

Renaming “Features” to “Skills”

A naming problem that made the product harder to explain than it needed to be.

Two different concepts no longer share a name

BEFORE	NOW
Coodra has two kinds of knowledge, and both were called some form of “Feature”: • Feature Packs — architectural blueprints for a whole module, given to the agent at the start of every session. • Features — small, self-contained instructions the agent fetches only when the conversation calls for them. The second is the industry-standard “Skills” pattern, but the shared name invited confusion in every conversation and every document.	The second kind is now called Skills throughout — the command group, the dashboard, and the tools the agent calls. Feature Packs keep their name, since that name was never the problem. • New command group: <code>coodra skill</code> (add, list, show, edit, index, remove, migrate). • A migration command moves an existing project's files to the new folder.

Nothing that used the old names broke

BEFORE	NOW
—	Every previous name still works: • The old command remains as an alias. • The three renamed agent tools still answer to their old names, while presenting only the new ones — so the advertised tool count is unchanged at 20 and existing agent configurations keep working. • Dashboard links keep their existing addresses, with the new ones redirecting. • Projects using the old folder are read from it; nothing is moved unless the user runs the migration.

Problems found by using the product

After the six phases, the whole product was exercised end to end — every command, every dashboard page, and a fresh graph of its own codebase. These were found by use, not by reading code.

Sessions still in progress were being cancelled

BEFORE	NOW
Coodra tidies away sessions that ended without closing properly. It decided a session was abandoned purely by its age, so any session running longer than 30 minutes was cancelled while the developer was still working. A second problem was worse: opening a second terminal on the same project immediately cancelled the first terminal's session, with no age check at all. This was found first-hand — the quality sweep's own working session was cancelled underneath it.	A session is now judged by activity, not age: as long as it is still recording work, it is left alone, however long it has been running. A genuinely abandoned session stops recording and is still tidied away as before.

The command menu listed commands it then refused to run

BEFORE	NOW
In the interactive terminal interface, six of the newer command groups appeared in the menu but produced “not a command” when selected. A related defect meant that fixing only the menu would have caused those commands to shut down the whole interface when run.	All listed commands run. An automated check now compares the menu against the real command set, so a command can no longer be added without appearing correctly.

A project could disappear from the dashboard without explanation

BEFORE	NOW
The dashboard deliberately shows only the projects belonging to the current organisation — correct behaviour, which prevents one team's work appearing in another's view. But when a project fell outside that scope it simply vanished, while the command line still reported it as registered. The user was given no way to understand the discrepancy.	The filtering is unchanged; the silence is not. Listings now state how many projects are hidden and why, and point at the command that shows the current organisation.

Smaller usability corrections

Individually minor, and each one a place where the product made a user work harder than necessary or told them something untrue.

BEFORE	NOW
A warning that was not real. The status command reported a “pending blocker” that was in fact the explanatory header of an otherwise empty notes file.	A blocker is reported only when a real, unresolved entry exists.
An unreadable list. In the list of recorded sessions, long identifiers overran their column and collided with the status beside them.	Columns hold their alignment however long the identifier.
A broken instruction inside generated files. Every newly created skill contained a reference to a tool that does not exist under that name — so an agent following it would fail.	The generated file names the correct tool.
Four options that did nothing. Options documented as “reserved”, which a script could pass and reasonably expect to have an effect.	Removed. An option that does nothing is worse than no option.
A required value that was ignored. Switching organisation demanded an argument that was only repeated back to the user.	The argument is optional; the choice is made in the browser, where it always was.
A dead end after success. Starting the services printed port numbers and stopped, leaving a first-time user without the dashboard address.	Prints the dashboard address and the next command to run.
An empty dashboard with no explanation. A new user with no recorded sessions saw a value dashboard reading <code>\$0.00</code> beside a column of dashes.	Explains that figures appear once the first agent session has been recorded.

Documentation and internal clean-up

The product's instructions to AI agents had drifted from the product

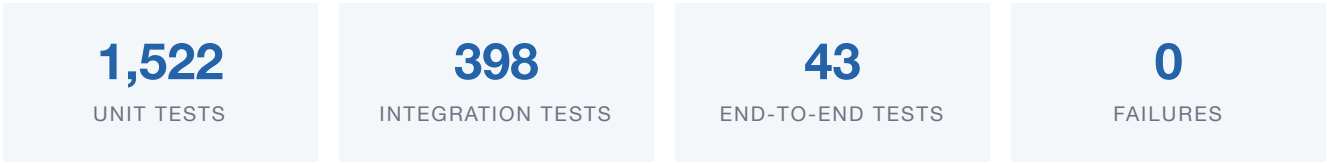
BEFORE	NOW
Coodra's own instruction documents are read by AI agents at the start of every session, so an error in them misdirects real work. The rename and an earlier expansion of the tool set had been applied to the software but not to those documents. The instruction file simultaneously told agents to call tools by names the product no longer advertises and stated that any tool not advertised does not exist — a direct contradiction. Tool counts were variously given as 9, 15, 17, and 26; the true figure is 20.	41 corrections across the agent instructions, the architecture specification, the README, the developer guide, and the public documentation page. Every tool name, count, and file path now matches the software. The example configuration file was also corrected: settings that nothing reads were removed, and two that are genuinely required were documented for the first time.

Unused code removed

BEFORE	NOW
Superseded helpers, an abandoned error type, two unused dashboard queries, and an unused third-party dependency remained in the codebase — each a thing a future reader would have to understand before concluding it did not matter.	22 items removed, each verified unused across the whole codebase first. One database table was deliberately <i>kept</i>: its migration has already been applied to live databases, so removing it is a separate, deliberate decision rather than clean-up.

How this was verified

Each phase was completed, tested, and validated on a real machine before the next one started — no phase was left partially done.



What each layer proves

- **Unit tests** — individual pieces behave correctly in isolation, including a test for every defect described in this document, so none can silently return.
- **Integration tests** — the pieces work together against real databases rather than stand-ins.
- **End-to-end tests** — a complete agent session runs through the whole product. This layer caught a defect the phase testing missed: a tool description had grown past its documented size limit.
- **Live validation** — every command was run and every dashboard page opened on a real machine, with a fresh graph built from this product's own codebase (13,982 nodes). The most serious defect in the quality sweep was found this way and not by any test.

Release package

- Installation package built and verified: **25.7 MB**, containing all five internal services and the dashboard.
- Automated pre-release checks confirm the dashboard is bundled and current — a safeguard added after a previous release shipped a stale copy.

Remaining actions

OWNER	ACTION
Publisher	Sign in to the package registry and publish. The package is built and verified; publishing requires credentials that are not available to an automated process.
Each machine	Restart the background service after upgrading, so the corrected session handling takes effect. Machines already running keep the previous behaviour until they do.
Backlog	Two optional improvements identified during the sweep and deliberately not bundled into it: a health check for a configuration mismatch found on the validation machine, and shorter session identifiers.